Terminology-Constrained Machine Translation for Enterprise Software Documentation

Ivan Pylypenko **Robin Schukrafft** **Ilnur Gayetov**
Technical University of Munich
{ivan.pylypenko,robin.schukrafft,ilnur.gayetov}@tum.de

Abstract

Translating domain-specific documentation takes more than fluency: a source term often has several plausible target-language forms, but only one matches what an organization has actually standardized on, and a system that picks a different one silently breaks terminology consistency across a large body of documentation. The WMT25 Terminology Translation Task frames this as a sentence-level problem: each source sentence carries a required set of source→target term pairs, and systems are scored on whether their output actually surfaces those forms, not only on fluency. The shared task's sentence-level track is itself built from SAP enterprise software documentation. Since it provides no training split, we supplement it with a separate, larger SAP post-edited corpus for fine-tuning. We compare two complementary ways of injecting terminology knowledge: conditioning GPT-4o-mini and Qwen2.5 models on a prompted term list at inference time, and adapting Qwen2.5 via LoRA fine-tuning. We further vary how the term list itself is built – from a human-annotated list to LLM-expanded, LLM-cleaned, and dictionary-mined variants – to separate list *coverage* from list *precision*. Injecting the correct term list raises GPT-4o-mini's macro BLEU from 39.6 to 46.1 and chrF from 65.2 to 72.0. An unrelated-pairs control list raises BLEU only to 42.4, showing the gain comes from term *correctness*, not merely from following an injected list. How the list is built matters as much as whether one is used at all: LLM-expanded and dictionary-mined lists give the largest BLEU/chrF gains, while an LLM-cleaned, domain-filtered list gives the largest term-accuracy gains. LoRA fine-tuning closes most of the fluency gap to GPT-4o-mini (51.4 vs. 47.5 BLEU), but GPT-4o-mini remains more reliable at realizing the required terms (74.8% vs. 69.8% term accuracy). Code and data: https://github.com/vnpnko/terminology-translation.

1 Introduction

Translating documentation is rarely difficult at the level of grammar. The difficulty is choosing the right target-language form for a domain-specific term: an English source word often has several plausible translations, but a company or product has typically standardized on only one, used consistently across its screens and manuals. Generic machine translation systems tend to pick a fluent alternative instead. It reads well in isolation, but it silently breaks terminology consistency across a large, interlinked body of documentation.

We treat this as a terminology-constrained machine translation problem, following the sentence-level framing of the WMT25 Terminology Translation Task (Semenov et al., 2025): each source sentence is paired with a small set of required source→target term pairs, and a system is scored not only on overall fluency but on whether its output actually surfaces the required target-language form for each term. A task record therefore consists of an English source sentence, a target-language reference, and its associated term pairs (Section 3.2 gives the full format).

The shared task's sentence-level track is itself drawn from SAP enterprise software documentation (Berger et al., 2025), where an English word like "request" can be translated into German as either the generic *Anfrage* or the terminology-correct *Antrag* (Figure 1). Section 3.1 describes how we use the shared task's own materials alongside a separate SAP corpus for training, since the shared task itself provides no training split.

Our strategy is to compare two complementary ways of injecting terminology knowledge into a translation system: conditioning a model at inference time on a prompted term list, and adapting an open model's weights via LoRA fine-tuning (Hu et al., 2022), evaluating both against a closed frontier model. Within prompt conditioning, we further

vary how the term list itself is built – from the human-annotated list shipped with the evaluation set, to versions expanded or cleaned by an LLM, to a list mined from a separate, related corpus – to separate the effect of list *coverage* from list *precision*.

We ask three questions:

1. **Does terminology conditioning help?** Injecting a term list into the prompt should raise translation quality and, more directly, the rate at which required terms appear in the output. But the model could simply be responding to *any* injected list, not to term *correctness* specifically. We test this with a control: an unrelated-pairs list evaluated alongside the real term list.

2. **Does how the term list is built matter?** We compare the original human-annotated list against an LLM-expanded version, an LLM-cleaned (domain-filtered) version, and a list mined from a separate, related corpus, to see whether list *size* and list *precision* trade off against each other.

3. **Can LoRA fine-tuning close the gap to a closed frontier model?** We fine-tune open Qwen2.5 models (Qwen Team, 2024) with LoRA on SAP documentation data and compare against GPT-4o-mini, with and without prompted terminology at inference.

2 Related Work

Terminology-aware machine translation covers a range of techniques, including constraint-aware training, placeholder substitution, and glossary injection at inference time. The WMT25 Terminology Shared Task (Semenov et al., 2025) found that terminology conditioning is especially useful for already-strong systems, and that evaluation must look beyond generic n-gram overlap to determine whether required terms actually appear in the output – we adopt that as a requirement throughout this report, treating term accuracy as a first-class metric alongside BLEU and chrF.

Prompt-based glossary injection conditions a model without changing its weights, in the same spirit as broader in-context learning with LLMs. He et al. (2024) instead have the LLM itself extract and translate keyword pairs to inject as prompt guidance. Their ablation shows this self-generated guidance still helps, but their own analysis finds the generated translations match the human reference term only about half the time – underscoring why, as we investigate in RQ2, how a term list is built matters. LoRA fine-tuning (Hu et al., 2022) is the complementary route when in-domain parallel data is available, adapting model weights directly instead of relying on inference-time conditioning. On the same SAP post-edit data we use, Berger et al. (2025) instead combine supervised fine-tuning with preference optimization over post-edits. Prior terminology-constrained MT and LoRA fine-tuning work each address part of this problem individually, but neither compares how the term list is *built* against fine-tuning the model itself. We compare both routes on the same SAP documentation data under the same prompting setup and metric suite, focusing not on a new decoding or adaptation method but on a controlled comparison of term-list construction and adaptation strategies for one enterprise domain.

3 Data and Task

3.1 Corpora

We use two JSONL releases derived from SAP documentation, for three language pairs: EN→DE, EN→RU, and EN→ES.

dev_v1 is our main evaluation set: 500 sentences per language pair, held out as a test set throughout this report. It ships with a human-annotated term list, which we take unmodified as the `Original` term-list variant (Section 4.2) – the only artifact we carry over directly from the shared-task materials. Every other list variant is a construction built on top of it.

dev_v2 is prepared from the SAP `term_postedits` corpus (SAP SE, 2024) (2,000 lines per language). We use it as a training-set proxy for the WMT25 Terminology Shared Task's own training data, which is not publicly released. dev_v2 is therefore *not* the original shared-task release, only the closest available same-domain substitute. Section 4.3 describes the smaller training-set size actually used for fine-tuning, after overlap removal against dev_v1 and few-shot example reservation.

The official WMT2025 terminology shared-task materials (task documentation and two evaluation tracks) are also available in our released code, but are not themselves used as training data here, for the reason above.

3.2 Record Format

Each record contains an English source sentence, a target-language reference, a set of domain `proper_terms` (source→target term pairs the translation should respect), and a set of non-domain `random_terms`: an unrelated term-pair list used as a control condition to test whether a model benefits from term *correctness* specifically, rather than simply from following any injected list (Section 5.1). Figure 1 shows a real example record.

```
{"en": "The request is deleted.",
"de": "Der Antrag wird gelöscht.",
"proper_terms": {"request": "Antrag"},
"random_terms": {"deleted": "gelöscht"}}
```

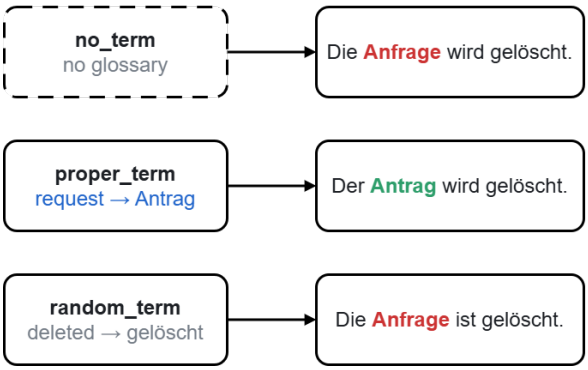


Figure 1: Effect of prompt conditioning on a single EN→DE example (dev_v1 index 440): `no_term` and `random_term` both mistranslate "request" as the generic *Anfrage*. Only `proper_term` produces the reference term *Antrag*.

3.3 Metrics

We define three metrics, computed per language pair and averaged (macro average) across the three pairs for prose-level comparisons: BLEU, chrF, and term accuracy.

BLEU (Papineni et al., 2002) and **chrF** (Popović, 2015) measure overall lexical similarity to the reference, but are weak signals of whether a specific required term form was actually used. We do not report COMET. The WMT25 task description listed it as an optional overall-quality metric, but the shared task's own findings paper ultimately relied on BLEU and chrF2++ instead, citing evidence that learned metrics like COMET can be less robust in domains underrepresented in their training data (Semenov et al., 2025).

Term accuracy (%) is the share of the provided `proper_terms` whose required target-language form appears in the model's output, computed against the term inventory of the condition being evaluated (the inventory differs across term-list variants, Section 4.2).

4 Methods

4.1 Prompt Conditioning

At inference, we translate with a short instruction and an optional glossary block. Three modes isolate the effect of terminology conditioning: `no_term` (translate without any terminology), `proper_term` (inject the domain term pairs for that sentence), and `random_term` (inject an unrelated, non-domain control list). Unless stated otherwise, runs use three few-shot examples per language. GPT-4o-mini is called through OpenRouter. Qwen models are run locally as instruction-tuned checkpoints.

4.2 Term-List Variants

Starting from the original human-annotated `proper_terms` shipped with dev_v1, we build three additional variants that change the glossary content while keeping the underlying sentences fixed. For the **expanded** list, an LLM is shown each source sentence together with its existing term pairs and prompted to propose additional in-context term pairs it judges relevant to the sentence, which are appended to the original list. For the **cleaned** list, an LLM is shown the expanded list per sentence and prompted to remove entries it judges overly generic or not domain-specific, keeping only the higher-precision pairs. For the **dictionary** list, we build a source→target term dictionary from dev_v2's own annotated pairs and match its entries back onto dev_v1 source sentences by exact string lookup, adding any new matches on top of the original list. These variants change both the size and the precision of the constraints supplied to the model, letting us separate the effect of list coverage from list quality. Table 1 reports the resulting term counts.

4.3 Models and Fine-Tuning

We evaluate GPT-4o-mini and open Qwen2.5-3B/7B-Instruct. For fine-tuning, we train LoRA adapters on Qwen2.5-3B/7B using a dev_v2-derived training set, held out from dev_v1 by removing exact-overlap lines and reserving few-shot examples (1,497–1,498 lines per language, depending on language pair). We sweep 1–3 LoRA epochs, and evaluate both zero-shot and few-shot prompt-

Variant	Terms	Construction
Original	1,590	human-annotated (dev_v1)
Expanded	2,573	LLM appends to Original
Cleaned	1,223	LLM filters Expanded[‡]
Dictionary	3,732	Original + dev_v2 matches[†]

Table 1: Term counts per variant, summed across all three language pairs. [†]Dictionary is additive: it adds new matches from a dev_v2-built term dictionary on top of the 1,590 Original terms, rather than replacing them. [‡]Cleaned applies a precision-oriented filter to the *Expanded* list (not to Original), which is why its count falls below Original's, not just below Expanded's.

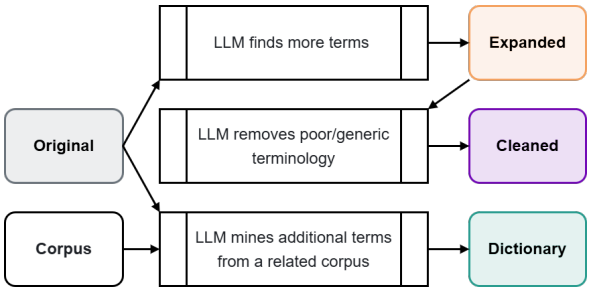


Figure 2: Construction of the four term-list variants: **Expanded** and **Cleaned** are derived from **Original** via an LLM expand-then-filter pipeline. **Dictionary** merges **Original** with terms mined from a related corpus.

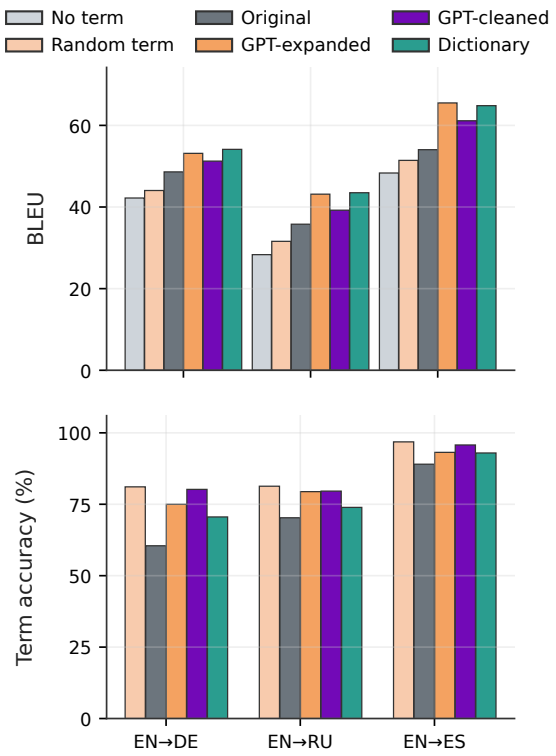


Figure 3: GPT-4o-mini BLEU and term accuracy by language pair on dev_v1: prompt modes (original term list) and term-list variants (`proper_term`).

ing at inference (with `proper_term` conditioning throughout). All fine-tuning runs are evaluated on the held-out dev_v1 test set.

5 Experiments

5.1 Terminology Modes Across Language Pairs

RQ: Does injecting explicit terminology constraints improve translation quality and term fidelity over no constraints, does an unrelated (random-term) constraint list act as a fair control, and does the benefit vary by language pair?

Injecting the real term list raises GPT-4o-mini's macro BLEU from 39.6 (`no_term`) to 46.1 (`proper_term`) and chrF from 65.2 to 72.0 (Figure 3, original term list). The random-term control also raises BLEU, but only to 42.4, well below `proper_term`, even though the model still realizes a large share of the *injected* random forms (86.4% term accuracy on the control pairs). This shows that following an injected list is not by itself what drives the quality gain: the content of the list matters, not just its presence.

The pattern is consistent inside each pair but not uniform: EN→ES is strongest on both BLEU and term accuracy in every mode, while EN→RU actually beats EN→DE on term accuracy despite trailing it on BLEU under `proper_term` – a single macro average therefore obscures which language pair benefits most from which effect.

Figure 3 also breaks out the term-list variants (Section 4.2) by language pair for GPT-4o-mini. One language-pair-specific effect holds across all three models: precision-filtering the expanded list into `cleaned` reliably raises EN→DE term accuracy, while the same filtering step is essentially a wash for EN→RU.

5.2 Terminology Modes Across Models

RQ: How does a closed frontier model compare to open base models on terminology-constrained translation, and does the ranking between models hold across term-list construction strategies?

Figure 4 reports macro-averaged BLEU and term accuracy for all three models under `proper_term` prompting, across all four term-list variants (Section 4.2). GPT-4o-mini leads both metrics in every variant, followed by Qwen2.5-7B, then Qwen2.5-3B. The model ranking is stable regardless of how the term list was built. Expanding the term list narrows the BLEU gap between GPT-4o-mini and

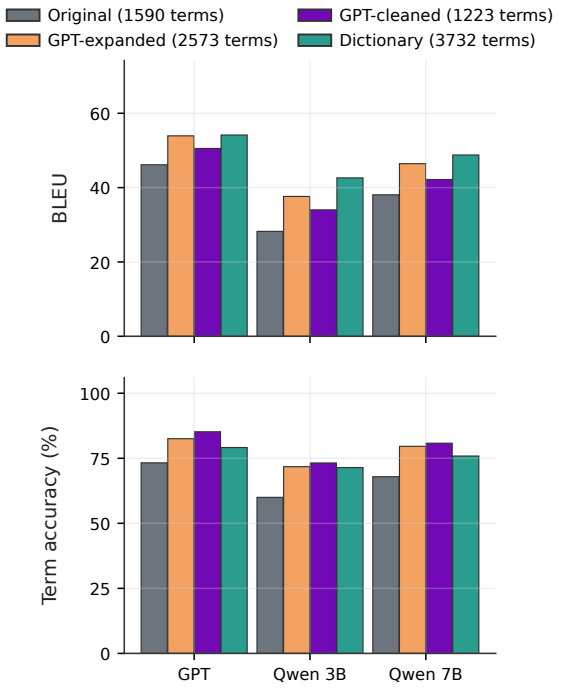


Figure 4: Macro-averaged BLEU and term accuracy by model and term-list variant, `proper_term`, dev_v1.

Qwen2.5-7B less than it narrows the term-accuracy gap: under `cleaned`, Qwen2.5-7B's term accuracy (80.8%) approaches GPT-4o-mini's (85.2%), while its BLEU (42.2) remains well below GPT-4o-mini's (50.5). This suggests the smaller model benefits disproportionately from a higher-precision list on term realization, not fluency.

Across variants, the same size-vs-precision trade-off observed for GPT-4o-mini in Section 5.1 holds for both Qwen sizes: `dictionary` and `expand` give the largest BLEU gains, `cleaned` gives the largest term-accuracy gains, for all three models. Moving from `original` to each model's best variant, the relative gain is also consistently larger for weaker models, on both BLEU and term accuracy alike. A better term list is therefore disproportionately more valuable exactly where raw model capability is lowest.

This size-vs-precision effect is not uniform across languages, however: EN→RU shows the largest proportional BLEU gain from terminology conditioning of any pair across all three models, a gap that does not shrink as model strength increases (Appendix D). On Qwen2.5-3B specifically, the small `original` list actually lowers EN→RU BLEU below the `no_term` baseline, a capacity-limited failure mode absent at 7B and for GPT-4o-mini.

5.3 LoRA Fine-Tuning

As background, dev_v2 – the proxy training set used below – scores consistently higher than dev_v1 across every prompt mode and metric (e.g. 51.2 vs. 46.1 BLEU under `proper_term`), consistent with dev_v2 simply being four times larger, not a distributional mismatch that would undermine its use as a training-set proxy. **RQ:** Does LoRA fine-tuning on dev_v2 close the gap between open Qwen2.5 models and GPT-4o-mini on terminology-constrained translation? All results in this subsection use `proper_term` prompting, zero-shot at inference (Section 4.3).

Figure 5 shows macro-averaged BLEU and term accuracy from the untrained zero-shot base through 1–3 LoRA epochs (full numbers in Appendix A). BLEU gains build steadily through training for both sizes (Qwen2.5-7B: 35.0 base → 51.4 at 2 epochs, Qwen2.5-3B: 16.6 base → 47.1 at 2 epochs) and largely plateau by the third epoch. Term accuracy moves far less than BLEU for Qwen2.5-7B (69.9% to 70.8% at 3 epochs) – fine-tuning mainly improves fluency, not term realization, which the base model already handles well. Qwen2.5-3B's term accuracy moves more (44.5% base to 68.0% at 2 epochs), since its untrained baseline starts substantially lower. A single LoRA epoch closes most of the size gap between the two models: Qwen2.5-3B comes within about five points of Qwen2.5-7B on both metrics after just one epoch, and stays there for the rest of training.

Figure 6 compares the best-performing open configuration, Qwen2.5-7B LoRA at 2 epochs, against GPT-4o-mini (full numbers in Appendix A). The tuned 7B model beats GPT-4o-mini on macro BLEU (51.4 vs. 47.5) and chrF (75.0 vs. 73.3), with the largest single-pair gain on EN→RU (45.7 vs. 37.2 BLEU). GPT-4o-mini still leads on term accuracy (74.8% vs. 69.8%) on every individual language pair, not just on average. The BLEU comparison instead flips by pair: Qwen2.5-7B LoRA leads on EN→RU and EN→ES but trails GPT-4o-mini on EN→DE, where GPT-4o-mini's advantage is really driven by term realization rather than fluency. We use the 2-epoch checkpoint as "best" because 3 epochs gives essentially the same BLEU/chrF for a small term-accuracy gain, though Appendix C notes this choice is not free of overfitting risk.

A leakage check (Appendix C) confirms this partly reflects train/test lexical overlap between dev_v2 and dev_v1: on a leakage-free subset,

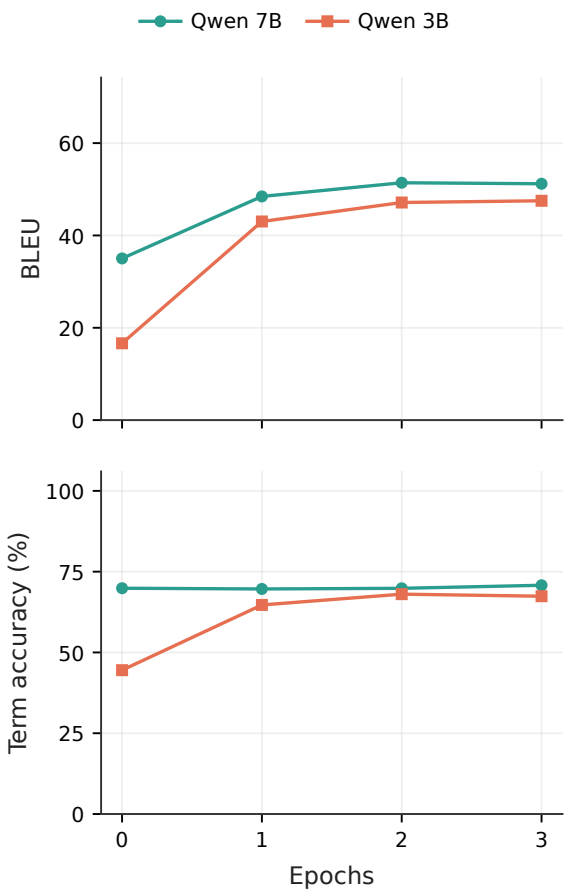


Figure 5: LoRA epoch ablation: trained on dev_v2, evaluated on dev_v1, zero-shot, `proper_term`, both Qwen2.5 sizes. Epoch 0 is the untrained zero-shot base.

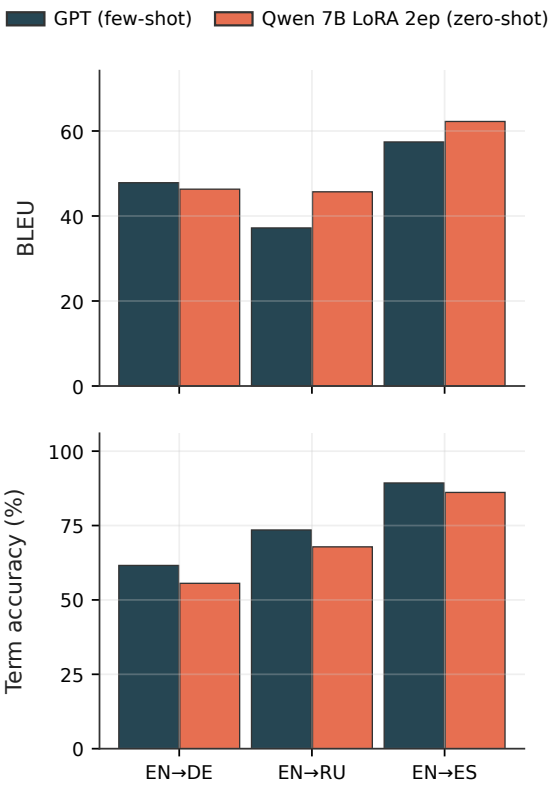


Figure 6: Best open configuration (Qwen2.5-7B LoRA, 2 epochs, zero-shot, trained on dev_v2) vs. GPT-4o-mini (few-shot), by language pair, evaluated on dev_v1, `proper_term`.

LoRA's BLEU edge over GPT-4o-mini shrinks from +6.9 to +0.8.

6 Discussion

Two tensions run through the results in Section 5.

First, generic and terminology-specific metrics disagree about what "better" means: the random-term control (Section 5.1) shows a model will realize whatever list it is given regardless of correctness, so term accuracy alone cannot certify conditioning is working – BLEU/chrF are needed to confirm the injected terms are correct. Conversely, among the real term-list variants (Section 5.2), dictionary and expanded lists win on BLEU/chrF while the cleaned list wins on term accuracy: if a deployment's priority is UI-label fidelity, not general fluency, BLEU is the wrong metric to optimize against.

Second, prompting and weight adaptation address different parts of the problem. LoRA fine-tuning (Section 5.3) recovers most of the fluency gap to GPT-4o-mini for Qwen2.5-7B, but moves term accuracy only modestly, since the base model already realizes injected terms well – fine-tuning mainly buys domain style, not term realization. This remaining gap reflects the model and prompting together. The tuned model still trails GPT-4o-mini on term accuracy specifically.

7 Conclusion

We studied terminology-constrained translation of SAP software documentation along three axes: whether terminology conditioning helps, how list construction shapes that gain, and whether LoRA fine-tuning closes the gap to a closed frontier model. Section 6 synthesizes what these axes mean in practice. In short, conditioning helps only when the injected terms are correct, list-construction strategy should follow the deployment's actual priority, and LoRA narrows the fluency gap to GPT-4o-mini without closing the term-accuracy gap.

Prompt-conditioning experiments here assume an oracle term list. A natural extension, left to future work, is having the model propose its terms.

Limitations

Our evaluation is narrow in scope: three language pairs (EN→DE/RU/ES) and a single enterprise documentation domain, so transfer to other products or domains is untested. The evaluation sets are also modest: 500 test sentences per language on dev_v1, and roughly 1,500 training sentences per language on dev_v2 after overlap removal. This limits statistical power. All results are single-run point estimates with no confidence intervals, so small gaps should be read as indicative, not confirmed.

Train/test lexical overlap is a concrete caveat, quantified in Appendix C. Some dev_v1 test sentences resemble dev_v2 training sentences, and the higher-overlap subset is inherently easier for every model tested, including one never trained on dev_v2. The fine-tuned model's gap is much larger, though, so some of LoRA's apparent BLEU advantage should be read as an upper bound, not one that would hold on disjoint documentation.

Most of our prompt-conditioning experiments assume an oracle term list is available at inference time. This isolates the effect of terminology control, but is not the deployment setting where terms must be extracted from the source with no oracle access. We have not yet validated our own extract-and-propose pipeline enough to report it here.

Finally, our term accuracy metric checks only surface-form presence of the required term. It does not capture morphological agreement or slot position, so it can over- or under-count correct terminology use in morphologically richer languages (notably Russian).

The "best" LoRA configuration we report (Qwen2.5-7B, 2 epochs) is chosen for the reasons given in Section 5.3 – a choice not free of mild overfitting risk past 2 epochs.

Acknowledgments

This work was carried out as a course project at the Technical University of Munich. We thank Dr. Marion Di Marco for supervision and feedback.

Author Contributions

Ivan Pylypenko: Implemented the LoRA fine-tuning pipeline for Qwen2.5-3B/7B using PyTorch and Hugging Face PEFT, and ran the epoch ablation, data-leakage honesty check, and few-shot ablation experiments, comparing the best configuration against GPT-4o-mini.

Robin Schukrafft: Built the GPT-proposed term pipeline and ran those experiments. Constructed the term dictionary and applied it to dev_v1. Ran GPT baselines on the dictionary setting. Implemented the matplotlib analysis scripts and result figures used in the poster and report.

Ilnur Gayetov: Set up the data-preparation pipeline for SAP `term_postedits` (JSONL task format, term-pair collection, initial term-expansion script) and restructured the project. Transformed shared evaluation into reusable modules and added evaluation runners for local Qwen and OpenRouter GPT-4o-mini.

References

Nathaniel Berger, Johannes Eschbach-Dymanus, Miriam Exel, Matthias Huck, and Stefan Riezler. 2025. Learning to translate ambiguous terminology by preference optimization on post-edits. arXiv:2507.03580.

Zhiwei He, Tian Liang, Wenxiang Jiao, Zhuosheng Zhang, Yujiu Yang, Rui Wang, Zhaopeng Tu, Shuming Shi, and Xing Wang. 2024. Exploring human-like translation strategy with large language models. *Transactions of the Association for Computational Linguistics*, 12:229–246.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.

Maja Popović. 2015. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisbon, Portugal. Association for Computational Linguistics.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

SAP SE. 2024. Software documentation data set for machine translation: term_postedits. GitHub repository.

Kirill Semenov, Xu Huang, Vilém Zouhar, Nathaniel Berger, Dawei Zhu, Arturo Oncevay, and Pinzhen Chen. 2025. WMT25 terminology translation task. Shared task website.

A LoRA Fine-Tuning: Full Result Tables

Table 2 and Table 3 give the full numbers behind Figures 5 and 6 in Section 5.3.

base	1 ep	2 ep	3 ep
Qwen2.5-7B, macro BLEU			
35.0	48.5	**51.4**	51.2
Qwen2.5-7B, macro Term Acc.%			
69.9	69.7	69.8	**70.8**
Qwen2.5-3B, macro BLEU			
16.6	43.0	47.1	**47.5**
Qwen2.5-3B, macro Term Acc.%			
44.5	64.7	**68.0**	67.4

Table 2: LoRA epoch ablation, zero-shot, dev_v1, `proper_term`. "base" is the untrained zero-shot checkpoint.

	Qwen2.5-7B LoRA (2ep)	GPT-4o-mini
BLEU	**51.4**	47.5
chrF	**75.0**	73.3
Term Acc.%	69.8	**74.8**

Table 3: Best open configuration vs. GPT-4o-mini, macro-averaged, dev_v1, `proper_term`, zero-shot.

B Few-Shot Ablation

Few-shot prompting has different effects before and after fine-tuning. At the untrained baseline (Table 4, top), it helps the smaller model far more than the larger ones: Qwen2.5-3B's macro BLEU roughly doubles (16.6 → 33.7) and term accuracy jumps 20 points (44.5% → 64.4%), while Qwen2.5-7B and GPT-4o-mini gain BLEU (35.0 → 38.4 and 41.9 → 47.5) but see a small term-accuracy *drop* (69.9% → 67.8% and 75.8% → 74.8%) – for already-capable models, the extra examples trade a little term precision for fluency. Once a model is fine-tuned, this effect largely disappears: holding LoRA at 1 epoch and varying zero-shot vs. few-shot at inference (Table 4, bottom) moves both model sizes by only a point or two in either direction (e.g. Qwen2.5-7B term accuracy 69.7% → 70.0%, BLEU 48.5 → 47.1) – fine-tuning and few-shot prompting appear to supply overlapping rather than additive signal.

C Data-Leakage Honesty Check

dev_v2 is used as a training-set proxy (Section 3.1) because the real WMT25 shared-task training data is unavailable. Since dev_v1 and dev_v2 are drawn from overlapping SAP documentation, some dev_v2 training lines are near-duplicates of dev_v1 test lines even after exact-overlap deduplication – fine-tuning on them could leak test signal and inflate LoRA's apparent gains.

	BLEU		Term Acc.%	
	zero	few	zero	few
Baseline (untrained)				
GPT	41.9	**47.5**	**75.8**	74.8
Qwen 3B	16.6	**33.7**	44.5	**64.4**
Qwen 7B	35.0	**38.4**	**69.9**	67.8
LoRA, 1 epoch				
Qwen 3B	**43.0**	42.2	64.7	**66.1**
Qwen 7B	**48.5**	47.1	69.7	**70.0**

Table 4: Few-shot ablation, macro-averaged, dev_v1, `proper_term`. Top: untrained baseline. Bottom: LoRA fine-tuned, held at 1 epoch.

Model	overlap	no_overlap	gap
Qwen base (untrained)	38.8	38.1	+0.8
GPT-4o-mini (control)	49.3	46.2	+3.2
Qwen LoRA (trained)	56.3	47.0	+9.3

Table 5: Macro BLEU on lexically overlapping vs. non-overlapping dev_v1 subsets (Qwen2.5-7B), balanced to 164 sentences/language/subset.

We split the dev_v1 Qwen2.5-7B test set into an `overlap` subset (near-duplicate training sentence exists, $\geq$50% token containment) and a `no_overlap` subset (no such duplicate), balanced to 164 sentences per language, and evaluate the untrained base model, GPT-4o-mini (never exposed to dev_v2), and the fine-tuned LoRA model on both. Table 5 reports macro-averaged BLEU. All three models score higher on `overlap` than `no_overlap`, and the gap grows with training exposure: negligible for the untrained base (+0.8), moderate for GPT-4o-mini (+3.2, despite never training on dev_v2 – an inherent difficulty difference between the subsets), and largest for the fine-tuned model (+9.3). On the leakage-free `no_overlap` subset, LoRA's BLEU edge over GPT-4o-mini shrinks from +6.9 (on `overlap`) to +0.8.

This should be read as leakage-inflation evidence *on top of* a real difficulty confound, not instead of it: the `overlap` subset's sentences are also substantially shorter (roughly 30–49% fewer words per sentence, depending on language pair) with a higher ratio of terms to words, while the total number of distinct terms per subset is nearly identical. This makes `overlap` sentences inherently easier for any model, which is exactly why even GPT-4o-mini (which never saw dev_v2) shows a gap. The fine-tuned model's substantially larger gap is the leakage signal on top of that shared difficulty effect.

D Term-List Variants by Language Pair: Qwen2.5-3B/7B

Section 5.2 reports that the model ranking (GPT-4o-mini > Qwen2.5-7B > Qwen2.5-3B) and the size-vs-precision trade-off between term-list variants hold across models, but only shows this macro-averaged over the three language pairs. Figure 7 breaks the same comparison down by language pair for both Qwen sizes at once, mirroring Figure 3's GPT-4o-mini view (Section 5.1).

Two patterns stand out beyond what the macro averages show. First, EN→RU sees the largest *proportional* BLEU gain from terminology conditioning across all three models: roughly a 50–65% relative rise from `no_term` to the best-performing variant, versus 30–47% for EN→DE/ES. Unlike DE/ES, this relative gain does not shrink much as model strength increases, suggesting Russian's morphological richness makes a good term list disproportionately valuable regardless of base model capability. Second, on Qwen2.5-3B specifically, the small human-curated `original` list *underperforms* both `no_term` and `random_term` on EN→RU BLEU: injecting the real but small glossary makes the smallest model's output worse than translating freely. This effect is absent at 7B and for GPT-4o-mini, consistent with a model-capacity limitation on making good use of a small set of forced substitutions in a harder target language.

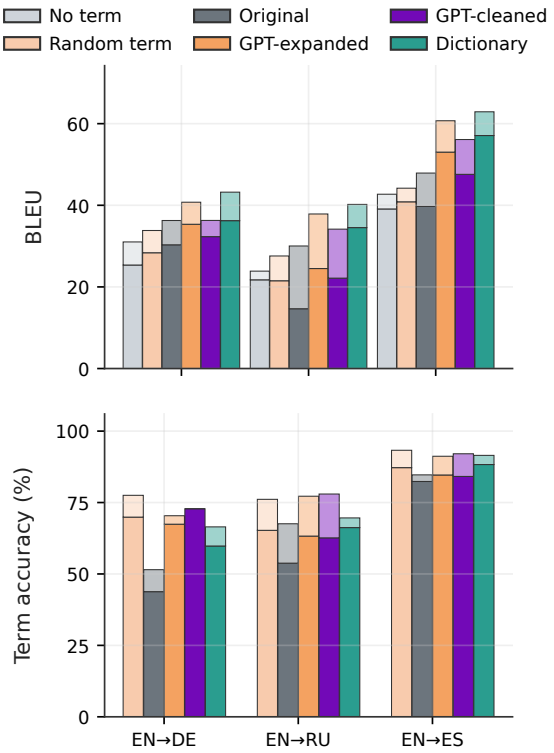


Figure 7: BLEU and term accuracy by language pair and term-list variant, both Qwen sizes: solid segment = Qwen2.5-3B, lighter segment = increment to reach Qwen2.5-7B, dev_v1.